

Agent 面试宝典

Learn-OpenClaw / interview 全章合订本

interview/ · Agent 面试宝典

clone 本仓库即拥有一份可直接用的面试宝典：每章「简答 + 扩展分析 + 追问预测」，观点型话术为主，所有判断对齐 Claude Code / Codex 的实际做法。

章节地图

章	文件	主题
01	01-agent-loop.md	agent 循环、自研轻框架、goal 长任务、工具做减法
02	02-tool-mcp-skill.md	Tool/MCP/Skill 统一视角、function call 原理
03	03-rag-不推荐.md	RAG≈VectorDB、agentic retrieval、选型
04	04-memory-context.md	压缩触发、tool_calls 边界、KV cache、MEMORY.md
05	05-复杂memory-不推荐.md	MemGPT/mem0/Zep/A-MEM 机制介绍 + 批判
06	06-压缩-从prompt技巧到模型能力.md	compaction 是 checkpoint、白马二、1M 窗口论
07	07-multi-agent.md	A2A 批判、subagent=上下文隔离、agent teams
08	08-eval-sandbox-harness.md	eval 分层、sandbox 信任边界、harness
09	09-eval-量化指标.md	完成率/token/耗时指标、定义完成、任务集建设
10	10-pi-mono-architecture-未验证.md	pi-mono 四模块拆解 + 架构图（未经实机验证）

主题分组：基础（01-02）→ 检索（03）→ context 工程三连（04-06）→ 多智能体（07）→ eval（08-09）→ 项目架构（10）。

01 · Agent 循环与框架

代码锚点: `core/node.py`、`examples/chatbot_with_tools`、`examples/agent_with_goal`

Q1: 讲讲你的 agent 是怎么工作的?

简答

一个 while loop: 调 LLM, 如果返回里有 `tool_calls`, 就执行对应工具、把结果以 `role=tool` 消息追加回 messages, 再调 LLM; 直到模型不再请求工具, 输出最终答复。我的实现里就是两个节点互相跳转: `ChatNode` 调模型, `ToolCallNode` 执行工具, action 路由决定下一步 (`examples/chatbot_with_tools/main.py`)。

扩展分析

其实 loop 本身是 agent 里最不性感的部分——真正的有意思的问题是 loop 的天花板在哪。说实话, 一个 loop 跑得好不好, 对 harness 的依赖远没有大家想象中大, 更多取决于基座模型的后训练: 模型在 post-training 阶段到底见没见过足够多的长轨迹、多轮工具调用数据。常规 loop 跑到几十轮之后, 上下文里堆满了工具输出, prompt 的分布早就漂移到模型训练时没怎么见过的区域了——对一个后训练不充分的模型来说, 这时候 harness 做得再精巧也不是瓶颈, 瓶颈是模型已经泛化不动了, 这一点 kimi-k3 系模型的技术报告里也有类似的观察。

所以我的工程结论是: harness 的职责不是「让模型变聪明」, 而是延缓漂移——压缩上下文、隔离子任务、少给无关工具, 本质上都是在帮模型待在它能泛化的分布里。想通这一点之后, 很多工程决策的优先级会自动排好。

追问预测

- 那具体怎么「延缓漂移」? → 三层手段: 上下文压缩 (见 04)、subagent 隔离 (见 07)、精简工具集 (见 Q5)。
- 你怎么判断模型「漂了」? → 轨迹层面的信号: 重复调用同一个工具、开始总结而不是行动、忘记最初目标。这些可以作为 eval 指标 (见 08)。
- loop 最多该跑多少轮? → 不设固定轮数, 设预算: token 预算 + 时间预算 + 无进展检测 (连续 N 轮没有新文件变更/新信息就中断)。

Q2: workflow、chatbot、agent 有什么区别?

简答

workflow = 节点按人写死的路径串联; chatbot = workflow 外面套一个「人输入→模型回复」的循环; agent = chatbot + tools, 路由由模型自己选。用一句话记: `agent = workflow + loop + tools`。本仓库 `core/node.py` 不到 60 行就把这三层都表达了。

扩展分析

其实 workflow 和 agent 的分界不是工程选型问题, 是基座模型能力的函数。workflow 的分支是人写的 (`node - "action" >> next`), agent 的分支是模型通过 `tool_calls` 选的——你该把多少决策权交给模型, 取决于模型的指令遵从和 zero-shot 泛化到了什么水平。

这个逻辑贯穿了整个 agent 架构的演进: 早期模型指令遵从差, 你必须用严格的 workflow 把它框死, 每一步都写死路径, 本质上是用代码结构去补偿模型能力的不足; 而现代模型的 zero-shot 能力已经非常强, 短 prompt 加宽松约束反而效果更好——你写一大堆 if-else 式的提示词去规训它, 反而把它限制在了更差的策略空间里, 甚至和它的后训练分布冲突。说实话, 这也是为什么后来 Claude 那边干脆全面转向 skill 的形态: 不再用冗长的 prompt 手把手教模型做事, 而是相信模型的泛化能力, 但是其实这个能力边界也是有变化的, 很难说什么时候哪个模型真的够用了

所以我的判断框架是: 先问这个决策模型在 zero-shot 下做得好不好, 好就放手 (loop + 短 prompt), 不好再问是 prompt 能补的还是后训练差距——prompt 能补的加少量约束, 补不了的 (支付、发布、删数据这种错了代价大的) 才写成 workflow。

追问预测

- 你的项目里哪里是 workflow、哪里是 loop？→ 单轮任务是 loop（chat↔tool_call 两节点循环）；`/goal` 是在 loop 外面再包一层 workflow 式的监督循环（见 Q4）。
- 纯 workflow 和 agent 效果差多少？→ 任务路径可枚举时 workflow 更稳更便宜；agent 赢在路径不可枚举的任务。选型依据是任务的「路径熵」，不是技术时髦度。

Q3：为什么自己造轻框架，不用 LangChain / Dify / Coze？

简答

agent 的核心语义（loop、tool call、message list）不到一百行就能写完，`core/node.py` 就是证明。LangChain 过度抽象、依赖重、还出过高危漏洞（CVE-2025-68664）；反过来看，业内最好的 agent——claude-code、cursor、kimi-cli、pi-mono——全是自研轻框架或者直接调 API。

扩展分析

其实这背后是个老问题：抽象该不该先于需求。LangChain 的问题在于它在 agent 范式还没收敛的时候就急着做抽象，结果范式每变一次，抽象层就错一次——你现在去调试，调用栈里一半是框架代码，根本不知道模型实际收到了什么 prompt。

说实话，agent 这种系统本身就充满不确定性，可调试性是最值钱的工程性质。我自己写框架层不到一百行，任何 bug 五分钟定位：模型收到了什么、工具返回了什么、哪一步错了，全部摊开可见。用框架省下的那点开发时间，远远付不起它带来的不透明性。Dify/Coze 这类低代码平台同理——demo 很快，但只要需求超出预设模板，你就开始和平台搏斗，而不是和问题搏斗。

追问预测

- 什么情况下你会用框架？→ 两天内要出 demo、或者团队完全没接触过 LLM API 时，框架是合理的脚手架；但产品化阶段我会逐步替换掉。
- 你们团队协作时轻框架不会有规范问题吗？→ 会，所以 node/flow 的接口要写得极小（exec 进、(action, payload) 出），约定大于框架。

Q4：agent 跑长任务会中途停下来、或者跑歪，怎么解决？

简答

加一个 goal 层（参考 Codex 的 `/goal` 和 Claude Code 的 hook 机制）：用户设目标后，每当 agent 自己停下来，一个 end hook 会拦截退出，交给一个专门的判断模型读轨迹、判定 goal 是否真的达成——没达成就自动注入提醒让它继续，达成了才放行。教学版最小实现（`examples/agent_with_goal`）用的是 `goal_complete` 工具，生产里我把它换成了 hook + 裁判模型。

扩展分析

其实这个设计的演进挺能说明问题。最小实现里「任务完成」是模型自己调 `goal_complete` 工具宣布的——但说实话这有个结构性的毛病：自证完成是利益冲突。模型天然倾向于过早宣布完成，这是后训练里偏好「给出完整、令人满意的答复」带来的副作用，让它自己当裁判等于默认了这个偏差。所以生产版本我把判断挪到了 agent 外面：end hook 拦截每一轮的退出，由一个专门的裁判模型来定夺——干活的和验收的必须是两个角色，这和工程里 QA 不能由开发兼任是同一个道理。

但说实话，裁判模型也不是一定就对了。我观察到 LLM 有时候会「嘴硬」：明明有问题，它在上下文里就是咬死说做好了，总结还写得像模像样——裁判只读上下文的话，照样会被骗。这本质上是信息问题，不是架构问题：上下文是干活模型的自述，自述可以是错的。所以更可靠的判据是让裁判看可验证的外部证据——测试输出、文件 diff、命令退出码，而不是只看模型的自我总结。但是这个就是一个非常复杂的工程问题了。claude code 里面的 goal hook 没有办法读文件，那被騙了是很有可能；但是如果你给他足够的权限去验证，我自己试过啊，他直接动手改代码了：我总不能给 goal hook 加个 goal hook 吧

追问预测

- 死循环怎么办？→ 其实有了 compact 机制之后，经典意义上的死循环（上下文撑爆、系统卡死）不太可能发生——超阈值就压缩，系统可以一直跑。真正要防的是原地打转：上下文还活着，但事情没进展。所以保险不是防循环本身，而是无进展检测（连续 N 轮没有新文件变更/新信息就中断上报）加时间预算。
- goal 提醒消息会不会污染上下文？→ 会，所以提醒消息在压缩时优先合并，只保留 goal 文本本身。
- 裁判模型用什么模型？→ 不需要最强模型，判断「做没做完」比「做完」容易得多，用小一档的模型控制成本；关键是喂给它外部证据而不只是对话记录。

Q5：工具设计多少个合适？怎么设计？

简答

越少越好。read / write / edit / bash 四个就是构建有效 coding agent 所需的全部（pi-mono 作者的结论），我在此基础上加了 grep / find / ls / search。Vercel 把 text-to-sql agent 的工具砍掉 80% 之后，成功率反而从 80% 升到 100%。

扩展分析

这个现象从模型角度很好解释：每个工具的 schema 都会占 prompt 空间，更关键的是，我们并不知道预训练的时候模型见过什么样的分布，很难说模型到底会什么。实际上我对现代模型的泛化能力有很深的怀疑：到底这个模型是真的有强大的泛化能力，还是只是预训练和后训练见得足够多？这个谁也说不清楚。但是我们能明确的是，考虑到现在大家都在蒸馏 Claude，而 Claude 就是这四个工具的拥护者，所以起码少量工具，模型一定是强力支持的，但是多了就不好说了

有意思的是，bash 一个工具在能力上是其他工具的超集——grep、find、ls 全是 bash 能干的。那我为什么还留着它们？其实也是因为现在的模型都在蒸馏 Claude。而且其实从模型行为来说，除非 sys prompt 强烈要求，否则 bash 命令还真就有干完一切工作的倾向

追问预测

- 工具报错怎么处理？→ 不重试、不吞错，把错误信息原样回传给模型让它自我修正——实测大部分参数错误模型一轮就能自愈。框架层的重试只用于网络类故障（node.py 里的 max_retries 就是干这个的）。
- 新加一个工具的决策流程？→ 先问 bash 能不能解决；能解决但调用频繁且参数易错，才做成专用工具。

02 · Tool / MCP / Skill

代码锚点: `tools/`、`tools/skill_loader.py`、`tools/mcp/`

Q1: Tool、MCP、Skill 有什么区别？

简答

本质上都是 tool——都是函数调用，区别只在位置和加载方式：Tool 是进程内函数调用；MCP 是远程函数调用（就是后端里的 RPC，只不过调用方是模型）；Skill 是本地函数 + 一份渐进式加载的说明书（SKILL.md）。

扩展分析

其实把这三个放在一起看，它们的发展史就是一部 prompt 经济学史。早期大家各写各的 tool，格式五花八门；Anthropic 定了 MCP 标准解决互操作，2025 年各家疯狂上 MCP 服务。但很快暴露出问题：每轮调用 LLM 都要把所有 MCP 工具的 name、参数、description 塞进 prompt——而这一次调用里大部分工具根本用不上。纯 token 浪费不说，还稀释注意力、拖慢首 token。Anthropic 自己在《Code execution with MCP》那篇 blog 里承认了这一点，给出的药方是渐进式加载 + 多用代码执行；后来发布的 Skill 就是这个思路的落地：默认只给模型一个名字和一句话描述，模型判断相关了再去读 SKILL.md，需要细节再去翻 reference 文件。

说实话，这个思想和操作系统里的按需分页一脉相承：上下文窗口就是内存，工具说明书就是按需换页的代码段。全量加载等于启动时把整个磁盘读进内存——没有操作系统敢这么干，但 2025 年一半的 agent 是这么干的。我项目里的 `skill_loader.py` 就是按这个思路实现的。

MCP 很多东西本身就经不起推敲：就为了「发现几个工具」这件事，搞了一整套协议、server、传输层、握手流程——工具发现真有这么难吗？一个 markdown 文件列清楚 API 用法，模型自己就会调了。而且你看 Anthropic：推完 MCP 不到一年，转头发 blog 说别这么玩、然后搞出了 Skill——发明协议的人自己都不玩这个了，留下一堆没人用的 MCP server。

工程上 MCP 还有个很难堪的问题：可调试性全面倒退。直接调一个 HTTP 接口，出了问题你能排——状态码、超时、SSL 证书、DNS，一层一层 curl 下去总能定位；MCP 后端 hang 住或者 dump 了，你就只能对着一个 stdio 进程干瞪眼，日志看它心情。所以我的态度很明确：能一个接口解决的事，绝不上协议。

追问预测

- 那 MCP 还有价值吗？→ 理论上有一——跨组织接第三方服务、不归你管的能力。但说实话，这个场景一个普通的 HTTP API 加一份写给模型看的 markdown 文档也能解决，而且更好调试。MCP 的真实价值更像「生态占位」，技术上它解决的问题远比它带来的问题小。
- Skill 和 RAG 有什么区别？→ 检索对象不同：RAG 检索的是「知识」，skill 检索的是「能力说明书」。但机制上确实是同一套惰性加载思想。

Q2: 你怎么决定一个能力做成 tool、MCP 还是 skill？

简答

高频、核心、参数简单的 → 内置 tool；能力在外部服务、不归我进程管的 → MCP；低频但流程复杂、说明书很长的专业能力 → skill。

扩展分析

决策变量其实就三个：调用频率、说明书体量、所有权。

- 高频的，说明书再短也得常驻 prompt——因为它每轮都可能被用到，惰性加载省不了钱，反而多一次「先读说明书」的往返。
- 低频的，说明书再详细也不能常驻——按需加载，不然每轮都在为十年用一次的能力付 token 税。

- 所有权是 MCP 的唯一硬理由：能力不在你进程里、升级节奏不归你管，那就只能走协议。

说实话，很多团队上 MCP 是「为了架构而架构」——本地一个函数能解决的事非要起一个 server，白付序列化、网络、进程管理三份成本。我的原则是 locality first：能本地不调远程，能代码执行不走协议。协议是最后的手段，不是起手式。

追问预测

- MCP 的实际接入体验如何？→ 协议本身不难（我仓库里有最小 server/client 示例），坑在生态质量：很多 MCP 服务的工具描述写得很随意，直接拉低模型选工具的准确率——接进来之前先改它的 description。

Q3: function call 的原理是什么？模型是怎么学会调工具的？

简答

宿主程序把工具 schema (name/parameters/description) 放进 prompt，模型输出结构化的 `tool_calls`，宿主解析、执行、把结果作为 `role=tool` 消息回传，模型接着生成。模型会调工具靠的是后训练阶段大量工具调用轨迹的训练。

扩展分析

这里有个被广泛误解的点：function call 不是 API 的魔法，模型的输出仍然只是文本——`tool_calls` 只是训练时约定好的特殊文本格式，由推理框架解析成结构化对象。这个认知的直接推论是：工具调用的可靠性本质上是模型后训练的结果。后训练里工具轨迹足够多、格式 reward 足够严的模型，调用就稳；反之怎么调 prompt 都救不回来。也不是救不了吧，但是精度就只能说……

所以工程上遇到模型乱调工具，我的排查顺序是：先换模型/检查 schema 是否写得有歧义，再考虑加 few-shot，最后才是写正则兜底——正则修得了一时，分布外的 case 永远修不完。当然兜底不能没有：我的 executor 解析失败时会把错误原样回传给模型让它自己修正，实测大部分错误一轮就能自愈。

追问预测

- tool 结果太长怎么办？→ 截断 + 引导模型用 grep/read 按需取细节。原则是「结论进上下文，细节留在磁盘」——和 skill 的惰性加载同一个思想。
- 并行 tool_calls 怎么处理？→ 独立调用可以并行执行，但回传消息必须保持 tool_call_id 一一对应、整组连续，不然一些模型 API 会直接报错（这个约束在我的 memory 压缩逻辑里也有体现）。

03 · RAG 与检索

Q1：讲讲 RAG？

简答

检索增强生成。现实里绝大多数「RAG 系统」只用到了「检索」：embedding 入库 + 向量检索 + 拼进 prompt，「增强」和「生成」本来就是 LLM 分内的事。所以业内有个说法：RAG $\approx$ VectorDB。

扩展分析

其实 RAG 这个概念已经有点过时了。2020 年提出的时候设想的是检索-增强-生成三段式 pipeline，好像三段各自都需要专门系统。但几年实践下来，「增强」和「生成」被证明就是 LLM 的原生能力，真正需要外部系统的只有检索——而向量库把这件事解决得很好。

更有意思的方向是 agentic retrieval：不预先建库，让 agent 拿着 grep/find 现场找。对代码库这种结构化、有精确符号可锚定的语料，grep 的准确率经常比向量检索高——因为 embedding 对标识符、文件路径、精确符号天然不敏感，而这些恰恰是代码检索的关键。我的项目里两条路都有：网络信息走 search 工具，本地代码走 grep/find。用什么检索方式应该由语料特性决定，而不是由「我刚买了个向量库」决定。

追问预测

- 什么时候向量检索不可替代？→ 三种情况：语义相似但措辞完全不同（「怎么退款」要匹配「退货政策」）、跨语言检索、大规模非结构化文档。共同点是「没有精确符号可锚定」。
- 面试官追问：那你说 RAG 过时，公司知识库问答怎么办？→ 知识库场景向量检索仍然成立（非结构化、措辞多变），我说的是「RAG 作为三段式架构概念」过时，不是说向量检索没用——这个区分要主动讲清楚，显得不极端。

Q2：embedding 和向量库怎么选型？

简答

原型期选 Chroma：嵌入式、零部署、API 简洁；embedding 直接用厂商 API（Kimi、智谱都有）。到了生产规模再考虑 pgvector / Milvus 这一档。

扩展分析

选型这事说白了是运维成本问题，不是算法问题。向量检索算法（HNSW 之类）各家的差异对业务效果的影响，远小于 chunking 策略和 embedding 模型选择的影响——但大多数人把时间花在纠结 Milvus 还是 pgvector 上，这是典型的「在能精确测量的地方使劲，而不是在真正重要的地方使劲」。

我的判断框架两句话：先选不会死的，再选运维得起的。Chroma 嵌入式零部署，原型阶段无敌；数据量真上来了，pgvector 是最顺滑的迁移——因为团队已经有 Postgres 的运维能力。引入一个新存储系统的真实成本是招聘、on-call 和故障演练，不是 benchmark 上那 5% 的 QPS 差距。

追问预测

- embedding 模型怎么选？→ 跟 LLM 选型同一家通常是起点（接口一致、账单合一）；真正拉开差距的是是否要微调——垂直领域术语多的语料，微调 embedding 的收益大于换任何向量库。

Q3：检索质量不好，怎么优化？

简答

按语义边界 chunking、加 overlap、metadata 预过滤、混合检索（向量 + BM25）、rerank 精排。

扩展分析

RAG 调优有个反直觉的经验分布：召回决定上限，rerank 决定下限，prompt 决定体感。大家最爱调 prompt，但 prompt 只能改变「模型用没用上检索结果」，改变不了「检索结果里有没有答案」——上限在你动手写 prompt 之前就已经定了。

混合检索的价值在于两种方法的失败模式正交：向量检索输在精确符号，BM25 输在语义改写，取并集是成本最低的提升。rerank 是用一个更强的模型把粗排 100 条精排到 5 条，本质是拿延迟换精度，值不值看场景。但说实话，这一套都做完还不行的话，通常不是检索参数的问题，而是这个问题根本不该用静态 RAG 解——该上 agentic retrieval，让模型自己多轮找、边看边调整查询。静态检索是「一次猜中」，agentic 检索是「像人一样翻找」，后者的上限高一个量级。

追问预测

- chunk 大小怎么定？→ 没有银弹，由「答案的典型粒度」决定：答案通常是一段话就切段落级，答案跨章节就要更大 chunk + 层级检索。先badcase分析再动参数，不要反过来。
- 怎么评估检索质量？→ 攒一批「问题 → 标准答案所在文档」的对，算 recall@k。没有这个数据集，所有调优都是盲人摸象（和 06 的 eval 是同一个思想）。

04 · 上下文与记忆管理

代码锚点：[core/memory.py](#)、[examples/chatbot_with_memory](#)

Q1：上下文越来越长，你怎么管理？

简答

全量对话追加写入 `session.jsonl`；每次拿到 API 返回的 `usage.total_tokens`，超过上下文上限 90% 就触发压缩：较早的消息让模型压成一条摘要，最近 4 条原样保留；压缩边界会对齐 `tool_calls` 组，不把 assistant 的工具调用和对应结果拆到摘要两边（`core/memory.py`）。

扩展分析

其实上下文管理的本质是一门经济学：上下文是稀缺资源，每个 token 每轮要付两次钱——一次是 API 账单，一次是注意力稀释。所以压缩的核心矛盾不是「怎么压」，是「什么时候压」：压早了丢细节，压晚了爆上下文。我的实现是踩 90% 阈值才压，尽量晚压，因为摘要是有损压缩，丢掉的细节就永远回不来——模型后面如果想引用「三轮前那个文件的具体报错」，摘要里大概率已经没有了。

另一个不踩坑想不到的细节：压缩边界不能乱切。assistant 的 `tool_calls` 和对应的 `role=tool` 结果在 API 层面是绑定的，拆开了有的模型直接报错、有的会陷入混乱。

追问预测

- 为什么不用 sliding window 直接丢旧消息？→ 模型会忘记任务目标，长任务直接跑偏。摘要保留了「曾经发生过什么」的语义连续性——丢消息是丢数据，摘要只是换表示。
 - 90% 这个数怎么定的？→ 经验值，给「压缩这次调用本身」留出余量（压缩也要消耗上下文）。真正的讲究是触发信号用 API 返回的真实 token 数，而不是本地估算——估算和计费模型的分词经常差不少。
-

Q2：长期记忆怎么设计？

简答

压缩的时候顺便让模型判断「哪些信息值得长期记住」（用户偏好、关键事实、运行环境），写入 `MEMORY.md`；之后每轮把它拼进 system prompt。对话记忆管「刚才聊了什么」，长期记忆管「以后也可能有用的」。

扩展分析

长期记忆真正的难点不是存，是写准入：什么都往里写，`MEMORY.md` 很快变成第二个噪声源，反过来稀释 system prompt。我的取舍是让「压缩」这个动作天然充当筛选器——只有活得足够久、跨过了压缩边界的信息才有资格进长期记忆，存活时间本身就是重要性的代理信号，不需要额外设计打分机制。

说实话，现在业内对 memory 有点过度设计，动不动向量记忆、记忆图谱。但你看 `claude-code` 的实现，长期记忆就是一个 markdown 文件。为什么？因为记忆的最终消费者是 LLM，而 LLM 最擅长读的就是自然语言 markdown——给模型看的东西用数据库 schema 来组织，属于为 DBA 优化而不是为读者优化。记忆的读写格式应该围着模型的口味设计。

追问预测

- 记忆冲突或过期怎么办？→ 目前是追加式，生产上需要定期让模型自己做整理（dedupe、更新、删除）——说白了记忆系统也需要 GC，写入只是上半场。
 - 为什么长期记忆放 system prompt 而不是作为一条 user 消息？→ system 位置权重最高、最稳定（每轮都在前缀里，还能吃到 KV cache），而且语义上「关于用户的持久事实」本来就更接近系统设定而不是当轮输入。
-

Q3：为什么对话存储是追加写 jsonl，而不是每轮重写整个文件？

简答

已有消息保持不变、新消息 append，保证 prompt 前缀逐字节稳定 → 命中服务商的 prefix KV cache，更快也更便宜。

扩展分析

LLM 推理的成本大头在 prefill，而 prefill 是可以缓存的——只要你的 prompt 前缀和上一次调用一模一样，服务商直接复用 KV，跳过重算。你改了历史里任何一个字符，后面全部 cache miss，成本延迟一起涨。但是好像有些服务商也不光是前缀匹配。

这一条铁律能解释我代码里一串看起来「多此一举」的设计：为什么 session 是 append-only 的 jsonl；为什么压缩要尽量晚触发（压缩一发生，整个前缀作废，下一轮全量 prefill）；为什么工具结果按序追加而不是就地更新；为什么时间戳这种易变信息要放消息末尾而不是开头。说实话，prompt engineering 一半是写给模型看的，另一半是写给 KV cache 看的

追问预测

- 崩溃恢复怎么处理？→ jsonl 按行解析，坏行跳过；如果崩溃停在工具调用中间（assistant 有 tool_calls 但结果没写全），恢复时把这一整轮未完成的消息丢掉再开口——不然 API 会因为 tool_call 组不完整直接报错（`memory.py` 初始化里就是干这个的）。
- 多 session 怎么组织？→ 一个 session 一个 jsonl 文件，长期记忆全局共享。session 之间不共享对话历史——共享了反而破坏前缀稳定性。

05 · 复杂 Memory 系统：我用过，但不推荐

Q1：怎么看 MemGPT / Letta / mem0 这类记忆框架？

先讲清楚它们是什么

- MemGPT (2023, UC Berkeley 论文)：核心创意是把操作系统的虚拟内存思想映射到 LLM——主上下文相当于 RAM，外部存储相当于磁盘，模型通过 function call 自己管理换页：上下文快满了就把一部分记忆「换出」到外部存储，需要时再「换入」。它主张 memory 不该由外部框架管理，而该由模型自己读写 (self-editing memory)。Letta 是 MemGPT 团队把这个想法产品化后的开源框架。
- mem0：托管/开源的记忆层。流程是 LLM 从每轮对话里自动抽取「事实」，写入时做去重和冲突更新（新事实覆盖旧事实），底层向量存储（另有图存储版本），对上就是 `add / search` 两个 API，主打「两行代码给你的 agent 加上记忆」。

简答

它们解决的是「记忆的存储和检索」，但 agent 记忆真正的难点是写入什么、何时读取、以什么形态进上下文——这三件事它们要么没解决，要么解决得很差。概念都漂亮，落到 coding agent 上都是负资产。

扩展分析

说实话我用下来觉得太垃圾了，而且是机制层面的垃圾，不是工程成熟度问题：

第一，错误率是相乘的。复杂 memory 是「抽取 → 存储 → 检索 → 注入」的四段流水线，每段都是模型或 embedding 在判，任何一段出错都污染最终上下文。尤其检索这一段：检索 query 是「当前对话」，而记忆是在另一个语境下写入的——embedding 对语境脱节天然不敏感，召回的记忆经常「语义像、时机错」。

第二，写准入是垃圾进垃圾出。mem0 那套自动抽取，本质是让另一个 LLM 判断「什么值得记」——判断错了没人审计，噪声记忆持续累积，记忆系统用得越久检索越差。这个性质和数据库完全相反：数据库越用越值钱，自动记忆库越用越像垃圾场。

第三，benchmark 严重刷分。LoCoMo 这类长对话记忆评测，塞进去的是多轮闲聊，和生产里「agent 跑了 50 轮工具调用后需要精确恢复操作状态」完全是两个任务。榜单上的分数外推到 agent 场景基本失效。

第四，做得最好的两家都不用。Claude Code 的长期记忆就是一个 markdown 文件 (CLAUDE.md)，Codex 就是 compaction——没有向量库、没有图谱、没有记忆框架。这不是巧合：模型足够强之后，「让模型自己维护一个 markdown」的效果超过一切外挂结构。复杂 memory 本质上是用工程结构补偿旧模型的弱点——模型一升级，补偿层就变成负债。和 workflow 框模型的逻辑一模一样（见 01 Q2），只是这次被框住的是记忆。值得一提的是 MemGPT 的「模型自己管记忆」这个方向其实是对的——但 Claude Code 用 markdown 文件把同一个思想落地得便宜一百倍，好思想不等于好框架。

追问预测

- 什么场景复杂 memory 是对的？→ C 端陪伴/客服：用户画像跨月累积、单条记忆价值低但量大、错检漏检无所谓。coding agent 里记忆错一条就是事故，场景的错误容忍度完全不同。
- mem0 有 dedup 和冲突更新机制，也不行吗？→ 机制本身还是 LLM 在判，判断错误会随时间累积，而且整个库对你都是黑盒——不能 git、不能 diff、不能审计的记忆系统，出问题时你连现场都 reconstruct 不了。

Q2：DAG / 图记忆 (Zep、Graphiti、A-MEM) 总该比向量强吧？

先讲清楚它们是什么

- Zep：记忆层产品，底层是它开源的时序知识图谱引擎 Graphiti。做法是：LLM 从对话里抽取实体和关系建成图谱，每条边都带时间区间；当新事实和旧事实冲突时，不删旧边，而是把旧边标记为「已失效」——于是事实的演化历史保留在图里，形成一个时序 DAG。检索是语义 + BM25 + 图遍历的混合。它主打解决「事实随时间变化」（用户换了地址，旧地址不该再被召回，但不该被忘记）。

- A-MEM (2025 论文)：灵感来自 Zettelkasten 卡片盒笔记法。每条记忆是一张结构化卡片（内容 + 关键词 + 标签 + 上下文描述）；写入新卡片时，LLM 自动检索相关旧卡片并建立链接，还会反过来「进化」旧卡片——根据新信息更新旧卡片的标签和上下文。卡片之间互相链接成网，检索走向量。思路是让记忆库像人的笔记系统一样自组织。

简答

理论上都成立：时序图解决「事实过期」，链接卡片解决「记忆孤立」。但构建和维护这些结构的成本与错误率，远超它们带来的检索收益。

扩展分析

图记忆的问题比向量记忆更深一层：它假设知识可以结构化三元组且不丢关键信息——这个假设在 agent 场景几乎不成立。agent 需要记住的是「我为什么做了这个决定、哪个文件改到一半、上次测试为什么挂」——这些是过程性状态，不是实体关系。硬塞进 (subject, predicate, object) 三元组，信息损失比摘要压缩还大。Graphiti 的时序失效机制解决的是「用户的地址变了」这类实体事实，可 agent 的核心记忆根本不是实体事实。

而且图的构建本身就是 LLM 抽取：抽错一条边，之后所有沿这条图的推理全部带毒；DAG 拓扑还要处理冲突边、时序失效、节点合并——你最终是在维护一个永远不一致的图。A-MEM 的「写入时进化旧卡片」听着优雅，但意味着每次写入都是一次全局 LLM 调用加一次图修改，写成本随库规模涨，而且旧卡片被改得面目全非之后你根本没法审计它原来长什么样。

说实话，同样的精力拿来维护一个文本文件，效果好得多。这类系统的论文都很漂亮，但落到生产有一个通用规律：你给模型看的东西，每多一层结构，就多一层「结构本身需要被模型理解」的成本——markdown 不需要被理解，图谱需要。给读者（模型）优化，不是给 DBA 优化（和 04 Q2 同一个原则）。

追问预测

- 那跨 session 的用户偏好怎么存？→ 见 04 Q2：MEMORY.md + 用压缩做写准入筛选。够用、可审计、可 diff、能进 git。
- 面试官说他们团队在用 Zep/Graphiti，怎么办？→ 不硬怼。先讲清楚它的机制（时序失效、混合检索）表明你真懂，再问场景——C 端聊天画像可以理解；然后讲为什么在 agent 场景不适用。能分场景讨论复杂 memory，比全盘否定更显老道。

Q3：如果面试官就让你现场设计一个 memory 系统（系统设计题）？

简答

分四层：working context（对话 jsonl + compaction）、episodic memory（session 级摘要）、semantic memory（用户偏好/关键事实，markdown）、procedural memory（工具/skill 说明书）。每层都用人类可读的文本存储，写入永远比读取保守。

扩展分析

系统设计题的关键是讲出分层背后的判断标准，而不是背层数。我的三条原则：

1. 生命周期管理比存储结构重要——记忆什么时候写、什么时候淘汰、什么时候合并，决定了系统一年后的质量；用什么库存，只决定第一个月的体验。
2. 所有层对模型透明——自然语言存储，检索优先用文件名/grep 这种可解释手段，embedding 只做兜底。模型读得懂、人审计得了。
3. 写入永远比读取保守——读错了是一轮的事，写错了是永久污染。

最后可以兜底一句：这套设计本质上就是把 Claude Code 的 CLAUDE.md 体系泛化到四层。和头部产品对齐的好处是，他们后续的实践演进你都能白嫖——这比自研一套「更精巧」的结构聪明得多。

06 · 压缩：从 prompt 技巧到模型能力

本章是 04 的进阶：04 讲的是「工程上怎么触发和执行压缩」，本章讲的是压缩本身是一种模型能力——通用模型直接做 compaction 是有分布缺陷的，头部产品（Codex 的专用压缩接口）已经把这件事下沉成了模型能力。内含一个可直接讲的自研段子：后训练一个类似 Codex 压缩接口的模型（白马二）。代码锚点仍为 [core/memory.py](#)。

Q1：直接用通用模型做 compaction，有什么问题？

简答

能用，但不对口。压缩是一个特殊任务：目标读者是「下一轮的自己」，不是人。它需要保留的是可执行状态——改了哪些文件、改到哪一行、上次为什么失败、下一步打算试什么。而通用 chat 模型写的摘要，是按「给人看」的分布训练的：流畅、概括、省略细节——丢掉的恰恰是最关键的操作状态。

扩展分析

这是我们实践里踩出来的认知：compaction 的质量标准不是文本指标，是「压缩之后任务还能不能接着做对」。一个好的 compaction 是 checkpoint，不是读后感。通用模型的后训练里，「总结」这个任务的奖励信号是面向人类读者的——人类读者要的是省略细节的流畅概括；但 agent 续跑要的是决策日志。两个优化方向是反的，你让一个被训练成「写给领导看」的模型去写「写给自己恢复现场用」的 checkpoint，分布天然就是拧的。

所以头部产品把压缩做成了专用能力：Codex 有专门的压缩接口。我们之前也尝试后训练了一个类似的压缩接口（内部叫白马二），其实也是一种快速压缩工具。

追问预测

- 白马二最大的坑是什么？→ 数据构造。「好的 checkpoint 长什么样」没有现成的 ground truth——人标的 checkpoint 未必是模型续跑最好用的。我们的解法是用下游成功率回采：同一条轨迹让模型写多版 checkpoint，各自接着跑任务，哪个续跑成功率高就留哪个当正例。本质上是把 eval 变成了数据飞轮（和 06 的思想一脉相承）。
- 值不值得自训？→ 看规模。prompt 一个强模型 + 精心设计的压缩模板，能拿到八成的效果；只有调用量大到压缩本身的成本和质量损失都可观时，自训才划算。

Q2：Claude Code / Codex 的 compaction 实践，有哪些可以直接抄？

简答

Claude Code：auto-compact + 保留最近消息 + CLAUDE.md 做长期记忆分离；Codex：专用压缩接口。共性是压缩是系统里的一等公民，不是事后补丁。

扩展分析

对齐两家的实践，有几条可以直接抄的决策：触发用 API 返回的真实 usage 而不是本地估算；压缩后是「摘要 + 最近 N 条」的混合结构而不是全压；长期信息和会话历史分开存（CLAUDE.md / MEMORY.md）；压缩调用本身也消耗上下文，阈值要留余量（这几条我在 04 里都已落地）。

但比具体决策更重要的是背后的演进方向：context 工程的历史，就是不断把「prompt 技巧」下沉成「模型能力」的历史。tool use 已经走完了这条路——早期在 prompt 里写 few-shot 教模型调工具，现在后训练直接教会；compaction 正在走同一条路——今天在 prompt 里写压缩模板

追问预测

- 那 context 工程未来会不会全部被模型内化掉？→ 压缩侧会，读写侧不会。记忆写什么、什么时候读，涉及外部状态和产品语义，这部分永远是工程问题。所以护城河在向两个方向迁移：eval（08）和状态/生命周期管理（05 Q3）。

Q3: 上下文窗口都 1M 了，压缩还需要吗？

简答

需要。窗口变大改变的是压缩的触发频率，不是压缩的必要性。

扩展分析

「上下文够大就不用管理」是一个周期性复活的幻觉，每次窗口翻倍都有人宣布 context 工程已死。三个理由它死不了：

1. 成本：上下文每轮全量计费，窗口大不等于你付得起把它塞满；KV cache 也不是无限大的。
2. 有效利用率：长上下文的中段信息召回率明显下降——lost-in-the-middle 在百万窗口下只是缓解，不是消失。塞得进 ≠ 用得上。
3. 分布漂移（见 01 Q1）：上下文越长，轨迹离模型的训练分布越远，泛化越差。窗口扩大反而可能加剧这个问题——以前撑到 100K 就被迫压缩回分布内，现在能一路漂到 900K。

说实话，上下文窗口和内存是同一个道理：内存从 640K 涨到 64G，内存管理没有消失，只是 GC 触发得少了。资源变便宜从来消灭不了资源管理，只改变管理的节奏。

追问预测

- 那什么真的会被大窗口消灭？→ 一部分 RAG 场景（中小规模语料直接全量塞入），以及浅层的多文档拼接任务。被消灭的是「因为塞不下而做的妥协」，不是「因为用不好而做的管理」。

07 · Multi-Agent / Subagent / Agent Teams

Q1：你怎么看 multi-agent？A2A 协议为什么没火起来？

简答

Google 推的 A2A (agent-to-agent 协议) 基本算失败了：交互复杂，多数场景效果打不过一个设计良好的单 agent，现实里也几乎见不到用 A2A 交互的产品。真正成立的 multi-agent 形态有两种：subagent (主 agent 派生子 agent 做子任务，只回传压缩结论，Anthropic 的 multi-agent research system 就是这个结构) 和 agent teams (同一套基础设施上的并行 agent，Claude Code 的 teams、Anthropic 用并行 Claude 写 C 编译器)。

扩展分析

其实 A2A 的失败和 MCP 的部分成功是一体两面：协议解决的是互操作问题，但 agent 的核心瓶颈从来不在互操作，在上下文质量。两个 agent 通过网络协议对话，交换的是自然语言——无损交换中间态的成本极高，于是实际落地时协议必然退化成「互相传压缩摘要」。那问题来了：既然交换的只是摘要，为什么不直接在一个进程里做？

而且 A2A 比 MCP 更尴尬：MCP 好歹解决过一个真实问题 (工具接入格式不统一)，A2A 连要解决的问题都基本是想象的——现实中你很难找到「两个对等 agent 分属不同组织、还必须实时协作」的场景。为一个不存在的场景定一整套协议，结果就是协议躺在文档里，没人用。

subagent 之所以成立，恰恰因为它不是「两个对等 agent 的协作」，而是主 agent 给自己做上下文隔离：把「需要大量工具噪音才能完成」的任务 (读 20 个文件找一个答案、搜 30 个网页写一段综述) 外包出去，子 agent 几十轮的工具调用完全不污染主上下文，最后只回传一段结论。说白了，subagent 是上下文管理工具，不是组织架构工具——用「公司部门、角色扮演」的隐喻去设计 multi-agent 系统，是目前最常见的误区，也是一堆 demo 很炫、实测更差的多 agent 框架的共同病根。

当然，得把话说精确：失败的是 A2A 这种跨进程、跨组织的对等协议型 multi-agent，不是 multi-agent 本身——Claude Code 的 agent teams 就是 multi-agent，而且效果拔群 (Anthropic 拿一队并行 Claude 写出了 C 编译器)。区分成败的判据不是「agent 的数量」，而是信任域和基础设施是否共享：teams 里所有 agent 跑在同一套原语上——git worktree 隔离代码、tmux 隔离会话、CI 做汇合仲裁——协作成本被现成的基础设施压到了最低；而 A2A 设想的是跨越信任域的协作，中间态交换的成本没有任何基础设施兜底，自然不成立。所以一句话：multi-agent 成立的条件是把协作成本工程化地压下去，A2A 想用协议把这个问题绕过去，绕不过去。

追问预测

- 那 Claude Code 的 agent teams 不就是 multi-agent 吗，这不是打脸？→ 不打脸，正好是我说的判据：teams 成立是因为所有 agent 共享同一套基础设施 (worktree、tmux、CI)，协作成本被工程压下去了；A2A 失败是想像跨信任域协作又没有基础设施兜底。同样是 multi-agent，差别在协作成本谁买单 (详见 Q2)。
- 什么时候 subagent 反而更差？→ 子任务之间强耦合、需要共享中间态的时候：切分意味着丢上下文，两个 agent 各知道一半，还不如单 agent 大上下文。判据是「子任务能否自包含描述」。
- A2A 完全没前途吗？→ 跨组织、跨信任域的 agent 协作 (你的 agent 调用别家公司的 agent) 协议还是有价值的——但那是 B2B 集成问题，不是 agent 架构问题。

Q2：Agent Teams (并行 agent) 是什么？和 subagent 有什么区别？

简答

目前最前沿的方向：摒弃主从结构，多个 agent 并行推进一个大工程。Anthropic 用一队并行 Claude 写 C 编译器，Cursor 也在做长时间自主编码。落地方式可以非常朴素：tmux 每个 pane 跑一个 agent + git worktree 隔离代码。

扩展分析

agent teams 和 subagent 的区别在于瓶颈从上下文转移到了协调。主从结构里主 agent 是串行瓶颈——所有子任务的结果都要经过它；teams 把任务真正并行化，吞吐上去了，代价是要解决冲突：两个 agent 同时改同一个文件怎么办？任务依赖怎么表达？

有意思的是，业界的答案朴素得出乎意料：全是现成原语——git worktree 隔离代码、tmux 隔离会话、文件锁隔离写入、CI 做汇合点仲裁。说实话这个方向最让我欣赏的一点是大家终于不发明新协议了。这也印证我一个判断：agent infra 的创新方向不是发明新抽象，而是把模型接入已有的成熟原语——bash、tmux、git worktree 都是久经考验几十年的东西，模型缺的不是新工具，是使用旧工具的权限和手艺。

追问预测

- 并行 agent 的任务怎么分？→ 按「可独立验证」切分：每个 agent 的任务要有自己的验收标准（测试、编译通过），汇合冲突才小。切不出独立验收标准的任务，强行并行只会把串行的痛苦变成合并的痛苦。
- 成本怎么控制？→ 并行 = token 成倍烧。适合「时间比钱贵」的场景（赶 demo、批量迁移）；日常任务还是主从结构性价比高。

Q3：你自己实现过 subagent 吗？怎么实现的？

简答

给主 agent 一个 spawn 工具：参数是子任务描述，内部新建一份独立的 messages 跑同一个 agent loop，把最终文本作为工具结果返回。几十行代码，因为 loop 是复用的。

扩展分析

实现上最关键的两个决策是给予 agent 多少上下文和让它返回多少。给太多，隔离失去意义；给太少，它做不好任务。实践的结论是：子任务描述必须自包含——子 agent 看不到主对话，你在 spawn 参数里没写的它一概不知；返回值必须是结论而不是过程——这一点要在子 agent 的 system prompt 里写死，不然它会把 20 轮的中间过程原样吐回来，隔离白做。

另一个决策是深度：子 agent 能不能再派生子 agent？我的取舍是默认不允许。嵌套派生的调试成本和 token 成本都是指数级增长的，真需要层级结构，说明任务拆分本身有问题——应该回去重新拆任务，而不是放任递归。限制 agent 的能力边界，往往比扩展它的能力更能提升系统可靠性，这个原则和工具设计（见 01 Q5）是相通的。

追问预测

- 子 agent 失败了怎么办？→ 失败信息原样回传给主 agent，由主 agent 决定重试、换策略还是自己做——主 agent 是唯一的决策者，子 agent 之间不允许互相感知，这样失败传导路径永远可追踪。

08 · Eval / Sandbox / Harness

Q1：怎么评估一个 agent 的效果？

简答

建任务集 + 可自动判分的指标（测试通过率、任务完成判定），每次改 prompt/模型/工具就跑回归。参考 Anthropic 的《Demystifying evals for AI agents》。

扩展分析

其实 evals 是 agent 工程里真正的护城河，也是最被低估的部分。模型三个月一换、prompt 天天改、工具不停加减——没有 eval，你根本不知道每次改动是变好还是变坏，所有「优化」都是体感玄学。我见过太多团队的 agent 改进流程是「改一下 → 跑两个例子 → 感觉变好了 → 上线」，这不叫工程，叫占卜。

agent eval 比传统 ML eval 难在一个地方：输出是轨迹不是标签。所以判分要分层：能程序判的（测试过没过、文件改没改对、命令成功没）坚决用程序判，便宜且确定；程序判不了的（答案质量、总结好坏）才用 LLM-as-judge，而且 judge 本身要定期人工抽检校准——judge 也是会漂的。说实话，提示词自动优化（DSPy 那一类）本质上也是 eval 的延伸：先有可信的指标，优化器才有意义，否则就是在优化一个噪声函数。追问预测

- 冷启动没有任务集怎么办？→ 从真实使用日志里挖失败 case。失败 case 是最值钱的 eval 素材——一个复现过的 bug 就是一个现成的测试。
- eval 跑不起（太贵/太慢）怎么办？→ 分层：每次提交跑小的 smoke 集（几分钟），nightly 跑全集。eval 和 CI 一样，贵在天天跑，不贵在规模大。

Q2：agent 的 sandbox（隔离执行）怎么做？

简答

Docker 够应对绝大多数场景：把 agent 的 bash/文件操作限制在容器里。只有对延迟极度敏感的场景才值得做专门的 agentic infra（microVM 之类）。

扩展分析

sandbox 的核心问题其实是信任边界画在哪。模型生成的代码本质是「半可信输入」——它不是恶意的，但它的错误边界你无法预知：它可能 `rm -rf` 错目录，可能 `curl` 到内网地址。Docker 把信任边界画在容器这一层，成熟、团队都会、故障模式众所周知。

真正需要自研 sandbox infra 的信号只有一个：延迟。agent 一轮任务可能起几十次隔离环境，容器冷启动几百毫秒乘上次数就成了体验瓶颈，这时候 microVM、gVisor 那一套才有讨论价值。我的原则是：为今天的威胁模型做隔离，不为想象中的 QPS 做优化——提前自研 sandbox infra 和提前买向量库集群是同一种病。

追问预测

- 容器里怎么防网络层面的风险？→ 默认无网或白名单出网——agent 需要的网络访问是可以枚举的（LLM API、包管理源、搜索 API），其余全禁。
- human-in-the-loop 和 sandbox 什么关系？→ 互补：sandbox 防「未知的坏」，审批防「已知但危险的合法操作」（比如 force push）。危险操作白名单 + 容器隔离，两层都要。

Q3: 听过 Harness Engineering 吗？怎么理解？

简答

OpenAI 在 2026 年初提出的概念：给 AI 搭更好的运行环境与工作流（比如 PRD→PR 自动化），让模型能力真正发挥到产品里。

扩展分析

说实话 harness 这个词目前还很虚——我看完 OpenAI 那篇文章也不能精确说出它指哪一项东西，不同人用它的意思都不一样。但这个词的流行本身说明了一件事：行业共识正在从「等更强的模型」转向「把现有模型用得更好」。

我的理解里，harness 就是 model 和 product 之间的整个转换层——loop、工具、上下文管理、eval、工作流编排，全都是 harness。这个视角下有个让人安心的推论：模型进步没有吃掉工程团队的价值，反而放大了它——同一个模型，harness 好坏能拉出数倍的产品差距。PRD2PR 是其中一个具象化：从产品需求文档自动推进到代码变更和 PR，核心难点不是模型聪不聪明，而是把 CI、测试、review 这些已有的工程设施接进 agent 的 loop——又回到那个判断：agent infra 的创新不是发明新东西，是把模型接进老东西。

追问预测

- 面试官：那你们团队的 harness 该怎么建？→ 从 eval 开始建（没有度量就没有 harness 的迭代依据），然后是上下文管理和工具层的打磨，最后才是工作流自动化。顺序不能反：先有度量，再优化。

09 · Agent 量化指标：怎么衡量、怎么定义完成

本章是 08 的展开，专攻面试高频区：任务完成率、token 消耗、耗时这些指标怎么定义、怎么采集、怎么优化。核心一句话：agent 的指标体系 = 结果指标（做没做对）× 效率指标（多大代价）× 过程指标（姿势健不健康）。

Q1：怎么衡量一个 agent 的好坏？都有哪些量化指标？

简答

三层指标。结果层：任务完成率（pass rate）、pass@k（k 次里至少成一次）、pass^k（k 次全成，衡量可靠性）。效率层：单任务 token 消耗、单任务成本、端到端耗时（p50/p95）、单任务工具调用次数。过程层：无效/重复工具调用率、回滚率（改完又改回去）、无进展轮次占比、压缩触发次数。

扩展分析

其实指标设计的第一课是 Goodhart 定律：指标一旦成为目标，就不再是好指标——这句话在 agent 身上是字面意义成立的，因为 agent 本身就是个优化器。你只考核完成率，它就学会「嘴硬」谎报完成（见 01 Q4）；你只考核 token 消耗，它就学会该查的资料不查、草草交差。所以指标体系必须是多维互相制衡的：完成率和效率一起考核，它才没法偏科。

第二层认知是：结果指标决定生死，效率指标决定商业价值，过程指标决定可调试性。完成率不行一切免谈；完成率相同的两家，单任务成本差三倍就是商业模式的差距；而过程指标（重复调用、回滚、无进展轮次）平时没人看，一旦完成率掉了，全靠它们定位是模型漂了、工具坏了还是任务集变了。说实话，很多团队只有第一层指标，agent 退化的时候只能干瞪眼——这和 MCP hang 住干瞪眼是同一个病：没有过程可观测性的系统，退化都是无声的。

第三层：pass@k 和 pass^k 的区分特别值得讲。demo 视频里都是 pass@k——跑十次成一次就能剪出惊艳效果；生产要的是 pass^k——用户不会容忍「十次里随机失败三次」。从 pass@k 到 pass^k 的距离，就是 demo 和产品的距离，这个距离全部要用工程填：重试、校验、goal hook、回滚。

追问预测

- 效率指标怎么算合理？→ 不只看绝对值，看性价比前沿：同完成率下比 token，同 token 下比完成率。还有一个我常用的衍生指标「成功任务的平均成本」（总消耗 / 成功任务数）——失败任务烧的 token 也要算进成本，这是最容易被忽略的口径。
- 耗时指标有什么讲究？→ 看 p95 不看均值：agent 耗时的尾部分布极长（跑歪的任务又慢又错），均值会把这些藏起来。p95 同时是体验和成本的双重先行指标。

Q2：「任务完成」到底怎么定义？谁来判定？

简答

分三层判定：能程序验证的坚决用程序（测试通过率、文件 diff、命令退出码、schema 校验）；程序判不了的用 LLM-as-judge + 明确 rubric；judge 本身定期用人工抽检校准。绝对不接受 agent 自报完成。

扩展分析

「定义完成」听起来是个小事，实际上是整个 eval 体系的地基——定义歪了，上面所有指标都在精确地测量错误的东西。我的原则是判定的可信度必须和判定的成本成反比地分配：程序验证最便宜最可信，能用尽用；LLM judge 贵且有偏，只用于程序够不着的开放任务；人工最贵，只用来做 judge 的校准采样。

程序验证有个反直觉的好处常被忽略：它把「完成」从模型的自述变成了环境的状态（和 01 Q4 goal hook 被嘴硬骗的问题是同一场战斗）。测试过了就是过了，模型嘴再硬也没用。所以设计任务集的时候，我会有意识地把任务改造成可程序验证的形式——「写个函数判断回文」比「写点有用的工具函数」好考一万倍。任务定义的质量，一半体现在它好不好判。

LLM judge 的坑也要讲：judge 有位置偏置、长度偏置（偏爱长答案）、以及和被测模型同源时的「惺惺相惜」偏置。所以 rubric 要写成二元判据的清单（有没有 X？有没有 Y？）而不是「打个 1-10 分」——离散的、可核验的判据比连续的评分抗噪得多；并且 judge 模型和被测模型尽量不同源，人工抽检 agreement 低于阈值就换 judge 或改 rubric。

追问预测

- 开放式任务（写方案、做研究）怎么判？→ rubric 分解 + 多 judge 投票 + 定期人工校准。接受它的噪声，但要知道噪声多大——用抽检估计 judge 的错误率，报告完成率时带上这个置信区间。
- RL 里也是这套吗？→ 同源。可验证奖励（verifiable reward）就是后训练的核心资产——工程上沉淀的判定器，换个身份就是 RL 的 reward function，eval 和训练在这里是打通的，这也是 eval 资产复利最高的地方。

Q3：eval 任务集怎么建？怎么防止指标越刷越水？

简答

从真实使用日志里挖失败 case 攒题（一个复现过的 bug 就是一道题），按难度和场景分类存放，定期补充新题；每次改动先跑几道核心题快速验证，全量题目留到每晚跑；另外专门留一批题「锁起来」谁都不许用来调优，只用来检验分数是不是虚高。

扩展分析

先解释一个关键概念：什么叫「指标越刷越水」（行话叫过拟合 eval 集）。就像学生反复刷同一套模拟题——分数越来越高，但高考不一定行。团队天天针对同一批题目改 prompt、修 bug，完成率涨到 95%，很可能只是把这套题「背」下来了：题目里埋的坑全都针对性堵上了，可用户随手丢来一个真实任务，照样翻车。防止这件事有三招，招招都是和「团队人性」作斗争：

第一招：留一批「锁起来的题」（行话叫 held-out set，保留集）。这批题平时谁也不许看、不许用来调优，只有阶段性验收时才拿出来跑一次。判别标准很简单：主题集分数涨、保留集分数也跟着涨，说明是真进步；主集涨、保留集不动，说明在背题，分数是刷出来的。

第二招：题目滚动换血。旧题通过率饱和（比如稳定 98% 以上）就「退役」——不再拿它指导优化，但保留在存档里定期回跑，防止老毛病复发（这个存档行话叫回归池）。同时不断补新题，让题集的难度始终贴着系统当前的能力边界——题太容易刷不出信息，题太难全是零分也没有区分度。

第三招，也是最难的：出题的和解题的不能是同一拨人。改 prompt 的人如果同时负责出题，他无意识中就会把题出成「自己的方案擅长做」的样子，eval 退化成自嗨。说实话，eval 团队的独立性和模型质量的正相关，我在不止一个地方见过。

还有个口径问题很多人栽过：报完成率必须说清题集的构成。同样 85%，「简单任务占八成的题集」和「专挑系统弱点出的题集」背后是两个完全不同的系统。题集构成就是指标的口径，口径不说清，数字毫无意义。

最后再补一个常见误区：不要「从公开 benchmark 下载一套就开跑」。公开 benchmark 一是可能早被模型训练时见过（等于开卷考试），二是它测的是「大家的 agent」不是你的 agent——你的题集必须长在你自己的失败分布上，从真实日志挖失败 case 为主，公开 benchmark 只做横向参照。失败 case 还有个隐藏好处：它是系统当前的最短板，修一个涨一分，比在已经 95% 通过的题海里加题高效得多。

追问预测

- eval 跑不起（贵/慢）怎么办？→ 分层跑。提交代码时只跑「冒烟集」（smoke test，借自硬件测试的梗：新设备通电先看冒不冒烟——挑几道最核心的题，几分钟出结果，验证没出大毛病）；每晚定时跑全量（行话叫 nightly）；正式发布前再加一轮人工抽查。eval 和 CI 一样，贵在天天跑，不在单次规模大。
- 模型升级时 eval 怎么迁移？→ 题目不动、判定器不动，新旧模型跑同一套题做对照再决定切不换——eval 基础设施的价值恰恰在于它不随被测对象变化，是你衡量一切变化的固定标尺。

Q4：指标收集上来之后，怎么用它指导优化？

简答

按「过程指标定位问题层 → 针对性修复 → 回归验证」的闭环走：完成率掉先看过程指标分层（模型漂移 / 工具故障 / 任务变化），修复后全量回归，不允许「跑两个例子感觉好了」就上线。

扩展分析

其实有了指标体系之后，优化 agent 的工作流会变得非常像性能工程：profile first, optimize second。完成率掉了，先按任务分层切片——是哪类任务掉的？掉的任务里过程指标长什么样？重复调用率高大概率是模型漂移或工具描述出了问题；无进展轮次高大概率是上下文管理出问题；工具报错率高那是真 bug。每一类失败都有自己的指纹，过程指标就是 agent 的火焰图。

优化优先级我也有个口诀：先修判定，再修分布，最后修 case。判定器有 bug 的话一切指标都是假的，最先排；某类任务整体塌方说明系统性问题（prompt、工具、模型版本），优先于个案；单点 case 最后修——而且修 case 的方式必须是改系统而不是往 prompt 里塞补丁，塞补丁就是在 prompt 里写死训练数据，维护两轮就腐烂。

说到底这一章的全部内容可以收敛成一句话：eval 不是项目末尾的验收环节，是驱动整个 agent 迭代的发动机——指标定义了什么叫好，判定器定义了什么叫对，任务集定义了什么叫难。这三样在手，模型随便换、prompt 随便改，你都知道自己在往哪走。

10 · pi-mono 架构剖析

本文件的定位：让你不用真的 clone pi-mono，也能把它的架构讲清楚。内容基于 pi-mono 仓库 ([badlogic/pi-mono](#) ，教程锚定提交 [3ffc2b43](#)) 的模块结构与作者博客《What I learned building an opinionated and minimal coding agent》整理。建议：跑通本教程第 8 章的改造之后，用本文件对照一遍你 fork 里的真实代码——讲的时候底气完全不同。

一句话总览

pi-mono 是一个 monorepo，共 7 个 package，但面试只需要讲 4 个：pi-ai（模型抽象层）、pi-agent-core（agent 循环）、pi-coding-agent（coding 工具集与会话）、mom（IM 接入与常驻运行壳）。其余 3 个（pi-tui、pi-web-ui、pi-pods）知道名字和职责即可。

架构图

```
flowchart TB
    subgraph 接入层
        Slack[Slack / IM] --> mom[mom<br/>IM 消息收发 · 会话路由 · 常驻进程]
    end

    subgraph 应用层
        mom --> coding[pi-coding-agent<br/>coding 工具集 · system prompt · 会话管理]
        coding --> tools[read / write / edit / bash<br/>极简工具集]
    end

    subgraph 核心层
        coding --> core[pi-agent-core<br/>agent loop · 工具调用调度 · 消息历史]
        core --> ai[pi-ai<br/>模型抽象 getModel · 流式输出 · 重试 · 多厂商统一接口]
    end

    ai --> LLM[LLM API<br/>Anthropic / OpenAI 兼容端点, 可换任意模型]
```

四个模块逐个讲

pi-ai —— 模型抽象层

- 职责：统一各家 LLM API 的差异（Anthropic、OpenAI 及兼容端点），对上暴露 `getModel(provider, id)` 这样的统一入口，处理流式输出、错误重试。
- 为什么独立成包：模型层是最易变的依赖——三个月换一次模型是常态。把它隔离成独立 package，换模型不动 agent 逻辑。
- 面试可讲的点：我们的改造就发生在这层的边界上——pi-mono 默认把模型写死成 `claude-sonnet-4-5`，我们通过环境变量覆盖 `model.id` 和 `model.baseUrl`，让同一个 codebase 可以跑在任何 OpenAI/Anthropic 兼容端点上（Kimi、DeepSeek、智谱……）。模型抽象层的价值不在抽象本身，在于「换模型是一行环境变量的事」。

pi-agent-core —— agent 循环

- 职责：agent loop 本体——调模型、解析 `tool_calls`、调度工具执行、维护消息历史，直到模型给出最终答复。

- 面试可讲的点：这层是整个项目里代码最少但语义最重的部分。它的实现印证了一个判断：agent loop 本身没有复杂度，复杂度全在「喂给 loop 的上下文质量」上——所以工程投入应该流向工具设计、上下文管理，而不是 loop 本身（见 01 Q1 的「延缓漂移」论述，可以和这里串着讲）。

pi-coding-agent —— coding 专用层

- 职责：在 agent-core 之上提供 coding 场景的全部特化：极简工具集（read/write/edit/bash）、system prompt、会话/工作区管理。
- 面试可讲的点：为什么 coding agent 变成了通用 agent——因为「读写文件 + 跑命令」这个能力组合可以表达几乎所有数字世界的工作。这一层的设计哲学就是作者博客的核心结论：四个工具就是构建有效 coding agent 所需的全部，剩下的复杂度都是负债。

mom —— IM 接入与常驻壳

- 职责：把 coding-agent 包装成一个长期运行的 Slack bot：收发 IM 消息、把消息路由到对应会话、保持进程常驻。
- 注意：mom 在 pi-mono 后续的 0ed0d434 提交中被移除，所以教程锚定 3ffc2b43。
- 面试可讲的点：这一层回答的是「agent 怎么从 demo 变成服务」——pm2 负责崩溃自重启和后台常驻，IM（Slack/飞书）负责触达用户。agent 产品化的最后一公里往往不是 AI 问题，是传统的服务化问题：常驻、重启、消息路由、会话隔离。

改造链路（把 pi-mono 变成 XXXClaw）

flowchart LR

```
A[fork pi-mono<br/>checkout 3ffc2b43] --> B[改 agent.ts<br/>env 覆盖 model.id / baseUrl]
B --> C[配置环境变量<br/>ANTHROPIC_MODEL_ID / BASE_URL / API_KEY]
C --> D[npm install & 构建]
D --> E[pm2 常驻运行 mom]
E --> F[Slack 接入<br/>和你的 XXXClaw 对话]
```

关键改动（教程第 8 章）：

```
// packages/mom/src/agent.ts, getModel 之下追加：
model.id = process.env.ANTHROPIC_MODEL_ID || "claude-sonnet-4-5";
if (process.env.ANTHROPIC_BASE_URL) model.baseUrl = process.env.ANTHROPIC_BASE_URL;
```

```
pm2 start packages/mom/dist/main.js --name mom --interpreter node -- --sandbox=host ./packages/mom/
data
```

面试官可能怎么问这个架构

- 「你和直接用 claude-code 有什么区别？」→ claude-code 是闭源产品，这是我自己掌控每一层的系统：模型可换（不被单一厂商绑定）、工具可裁剪、行为可 eval。讲的时候落到「可调试性 + 可替换性」两个词。
- 「为什么选 pi-mono 做底座而不是从零写？」→ 学习路径问题：从零写 loop 我在 Python 里已经写过了（本仓库 core/），但生产级 agent 的复杂度在工程细节——流式、重试、会话、常驻，这些直接站在一个被验证过的极简实现上学，比从零踩坑快一个量级。先学会造轮子（60 行），再学会选轮子（pi-mono），最后学会改轮子（XXXClaw）——这个三件套回答非常加分。
- 「最深的改动是什么？」→ 模型端点抽象（env 化）；如果做了工具增删或 memory 改进，讲那个，并带上动机（token 经济学 / KV cache，见 04）。